

DETECTION OF CIRCULAR TRADE

TEAM :

ES19BTECH11010

ES19BTECH11026

MA19BTECH11002

MA19BTECH11006

MA19BTECH11011

Abstract : There are many various ways to manipulate taxes, each with its own goals and levels of complexity. In this paper, we discuss circular trading, a particular kind of tax evasion. We define techniques for the detection and analysis of circular commerce in particular. To do this, we treated the entire system as a directed graph, with the traders serving as the nodes and the transactions connecting them as the nodes and directed edges, respectively.

I. INTRODUCTION

Fraud, unfortunately, has been inherent in our society from the dawn of time. It is primarily motivated by the unscrupulous desire of people to make personal benefits by exploiting the loopholes in the existing laws in a system. Some fraudulent practices are simpler to spot and investigate than others. On the other hand, certain fraudulent techniques are quite challenging to identify because of the intricate procedures needed to handle them.

In this article, we offer a methodical approach employing social network analysis to deal with circular trading, a sophisticated financial fraud that is pervasively prevalent in the commercial taxing system. Business entities engage in it with the goal of avoiding paying taxes that they are required to to the government. A business entity engages in circular trading when it creates bogus business firms and works diligently to organize with them to falsify the financial information supplied in their commercial tax return filing. This is a theft of Value Added Tax (or VAT) from the government.

A consumption tax known as value-added tax (VAT) is imposed on goods and services at every step in the supply chain where value is added, from the point of initial production to the point of sale. The cost of the product minus any expenses of components that have already been taxed at a prior step determines how much VAT the consumer must pay.

Let us consider an example to see how VAT works. Consider a manufacturer X, who buys raw materials from a producer P, on which VAT is charged for the materials. The VAT charged on these costs is called input tax. On the products produced after manufacturing, X sells these to the retailer say R, for which VAT is added to the selling price. The VAT charged on the selling price is called output tax. Then retailer R sells the products to the consumers who pay the VAT included in the final price of the product.

The net VAT liability manufacturer X should pay the government is given by :

$$\text{Net} = \text{output tax} - \text{input tax}.$$

The end user pays the entire VAT on the final price, but each party in the supply chain only pays the VAT on the value they have added. As a result of equally distributing the tax burden throughout all phases of production and consumption, the VAT system is a fair and effective method of tax collection.

Circular Trading

Circular trade is a fraudulent practice in which a number of businesses or people participate in a round-robin transaction plan in an effort to inflate sales or profits. It is often referred to as wash trading or round-tripping.

In circular trade, each participating company purchases goods or services from one of the other businesses in the group and then resells those same goods or services to a different business in the group for a comparable price. The transactions may include inflated costs, fictitious invoices, or other bogus paperwork to give the impression that real business is being done.

Circular trade has the effect of allowing the group of businesses engaged to give the illusion of higher sales or profits, which can be used to obtain loans, draw in investors, or raise stock prices. In actuality, though, no real value has been produced; rather, the money has just been distributed among the group of businesses.

II. PROBLEM STATEMENT

To detect circular trade given a set of buyers and sellers and the tax amount for each transaction or trade between the buyer and seller.

We need to detect circular trade using the Node2vec algorithm.

Hence, we need to detect circular trade given a list of transactions.

III. DESCRIPTION OF THE DATASET

We are given a csv file named Iron_dealers_data.csv which contains three columns, namely Seller ID, Buyer ID and Value.

Seller ID and Buyer ID give the id's of the seller and the buyer in the given transaction respectively, and the value denotes the tax amount value of that transaction.

The below table shows the dataset in the form of a DataFrame.

The total number of transactions/invoices given : 130535

The number of columns : 3

They are Seller ID : (int) value

Buyer ID : (int) value

Value : (float) value/ tax amount for the transaction

Given below is the head of the dataframe, that first ten rows of the dataset.

We can see multiple transactions taking place between the seller 1309 and buyer 1011. Similarly we can notice multiple transactions between various buyers and sellers. Thus for executing this we used a Multi-Directional graph.

So our data is transformed to a multi directional graph where there are edges from the seller_id to the buyer_id and weight of the edge being the tax amount value of the transaction.

```

↳ Number of rows: 130535
   Number of columns: 3
      Seller ID  Buyer ID    Value
0         1309      1011  1225513.0
1         1309      1011  1179061.0
2         1309      1011  1119561.0
3         1309      1011  1200934.0
4         1309      1011  1658957.0
5         1309      1011  1773165.0
6         1309      1011  1688506.0
7         1309      1011  1805376.0
8         1309      1011  1327164.0
9         1309      1011  1327539.0

```

IV. ALGORITHM USED : *Node2Vec*

Node embedding is the technique of encoding the nodes in a network as vectors in a high-dimensional space. Node2vec is a powerful algorithm to perform node embedding. It is especially helpful for graphs with complex, non-linear structures, such as social networks or biological networks.

The Node2vec algorithm works by generating random walks on the graph and learning a vector representation for each node based on the context of its neighbors in the random walks. This allows it to capture the local and global structure of the graph and identify similarities and differences between nodes.

For detecting circular trade using Node2vec, each seller/buyer involved in the trading scheme is represented as a node in our graph. Node2vec algorithm is used to get the vector representation of each node based on the patterns of trading between the companies.

If circular trading is occurring, we would anticipate to see a pattern of trading where each company in the group trades exclusively with other companies inside the group. As a result, the graph would take on an obvious clustering pattern, with the nodes denoting the companies engaged in the circular trading forming a tightly connected group.

We may find these clustering patterns and find instances of circular trade by applying the Node2vec algorithm to the graph and looking at the generated vector representations. This can be an effective technique for spotting financial fraud and locating businesses that are participating in unethical or unlawful behavior.

IV.(i) CODE EXPLANATION

Python libraries used : Pandas, Numpy, NetworkX, Node2vec, sklearn.cluster for kmeans clustering.

We create a MultiDiGraph named *trade_graph*. We open our file '*Iron_dealers_data.csv*' and parse each line to get *seller_id*, *buyers_id*, and the value. We add an edge between *seller_id*, *buyer_id* of the *trade_graph* with weight of the edge being *value*.

There are a total of 799 unique seller and buyer id's.

Thus the number of nodes in our graph = 799.

Now we use Node2vec from the node2vec library to generate random walks. We give *trade_graph* as our input, and *walk_length* as 30 and *num_walks* as 200. Now the random walks are generated. We then generate embeddings for each walk and convert it to an np.array. We now perform k-means clustering with 10 clusters. We form clusters using our embeddings from before.

The clusters formed are as follows:

```
[ '1837', '1132', '1293', '1310', '1326', '1114', '1475', '1419', '1103', '2000', '2017', '1379', '1377', '1533', '1030', '1511', '1950', '1722', '1388',  
[ '1747', '1535', '1009', '1146', '1406', '1553', '1145', '1008', '1646', '1743', '1026', '1739', '1028', '2126', '2111', '1010', '1942', '2104', '1027',  
[ '1832', '1488', '1944', '1503', '1862', '1575', '1729', '1917', '1545', '1581', '1617', '1582', '2034', '1939', '1959', '2071', '2190', '1859', '1914',  
[ '1259', '1163', '1748', '1172', '1189', '1723', '1405', '1344', '1191', '1389', '1181', '1168', '1813', '1470', '1065', '1484', '1390', '1469', '1471',  
[ '1210', '1668', '1099', '1666', '1090', '1930', '1679', '1159', '1541', '2073', '1754', '1678', '1162', '1201', '1116', '1956', '1100', '1927', '1088',  
[ '1309', '1147', '1039', '1045', '1018', '1007', '1076', '1048', '1205', '1057', '1032', '1061', '1023', '1327', '1056', '1003', '1002', '1004', '1051',  
[ '1393', '1005', '1042', '1074', '1034', '1220', '1075', '1350', '1276', '1228', '1138', '1089', '1264', '1595', '1101', '1079', '1104', '1135', '1809',  
[ '1875', '2184', '1633', '2031', '1780', '1562', '1757', '1640', '2028', '1977', '1632', '1629', '1580', '1157', '1877', '2162', '2027', '1634', '1975',  
[ '1011', '1659', '2085', '2088', '1658', '1656', '1500', '1990', '1690', '1961', '1962', '2166', '1891', '1501', '1855', '1857', '2108', '2152', '1505',  
[ '1568', '1256', '1801', '1222', '1017', '1623', '1774', '1766', '1431', '1701', '1517', '1528', '1447', '1529', '1215', '1696', '1598', '1261', '1597',
```

Total of 10 clusters are formed. (all nodes are not shown in the image)

Identifying circular trade within the clusters:

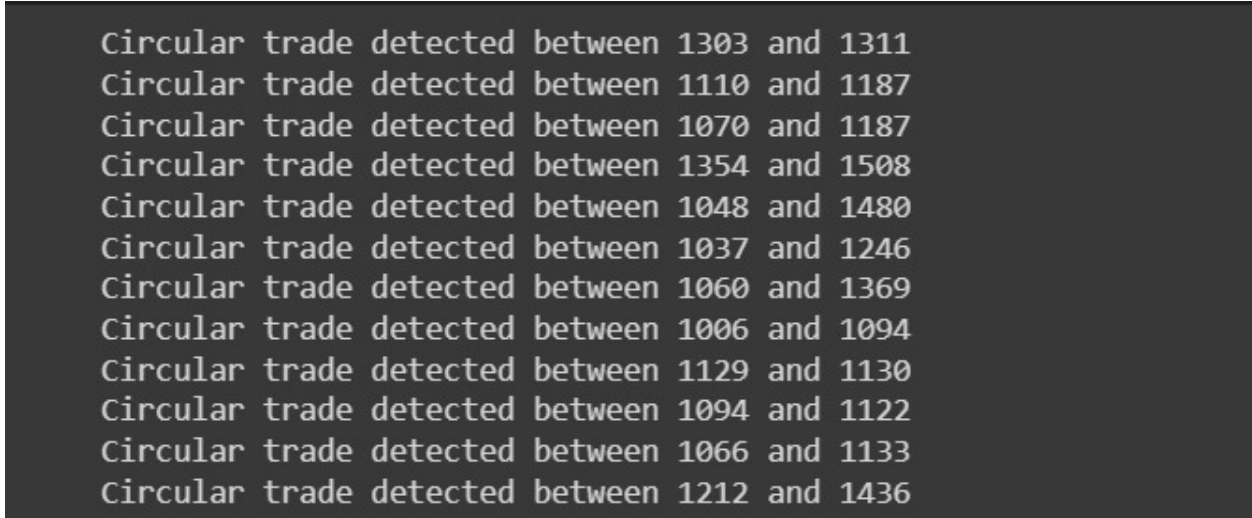
We iterate through each cluster. For every two nodes in the cluster, we check if an edge exists between them. If edge(s) exist between them then we compute the *total_value* of the transaction between the nodes. (sum of the weights of multiple edges between the two nodes).

Then we compute *node1_sales* and *node2_sales* which are total weights we get when we subtract incoming edge weights from outgoing edge weights of the nodes. Now we compare *total_value* to *node1_sales* and *node2_sales*, and it's greater than half of sales of any node, then we consider there is circular trade happening between node1 and node2.

Otherwise there is no circular trade.

V. RESULTS

The following image shows the result of our code:



```
Circular trade detected between 1303 and 1311
Circular trade detected between 1110 and 1187
Circular trade detected between 1070 and 1187
Circular trade detected between 1354 and 1508
Circular trade detected between 1048 and 1480
Circular trade detected between 1037 and 1246
Circular trade detected between 1060 and 1369
Circular trade detected between 1006 and 1094
Circular trade detected between 1129 and 1130
Circular trade detected between 1094 and 1122
Circular trade detected between 1066 and 1133
Circular trade detected between 1212 and 1436
```

We can conclude that circular trade is detected and is between the above mentioned traders as in the dataset given.